

AWS SECURITY MONITORING SYSTEM & REAL-TIME ALERTS

... By Pallavi Khadse

*Project category: Cloud Security
/ Zero-Trust architecture*

1. Introduction & Project Objective

In any cloud environment, keeping credentials, API keys, and database passwords safe is one of the most important security goals. This project shows how to build an automated *Security Monitoring System* on AWS to create a protective boundary around sensitive secrets.

The primary objective is to watch over *AWS Secrets Manager* in real time. Utilizing a *Zero-Trust approach*, any attempt to read or access our secret is immediately captured, analyzed, and processed. This infrastructure dramatically compresses the detection-to-remediation window from hours to seconds, ensuring security teams are notified the instant an access event occurs.

2. Architecture & AWS Services Used

- **AWS Secrets Manager:** It has secure, centralized storage and lifecycle management for confidential credentials.
- **AWS CloudTrail:** It tracks and records all API activity and user actions in the AWS account.
- **AWS CloudWatch Logs:** It is a centralized log engine that gathers and aggregates, retains, and query operational log data coming from *CloudTrail*.
- **AWS CloudWatch Alarms:** It is a real-time evaluation engine that monitors custom metric thresholds and executes automated actions triggering an alert if something is breached.
- **AWS SNS (Simple Notification Service):** It is a fully managed Pub/Sub messaging service that instantly delivers multi-destination alerts to configured subscribers. I will create it to send the alarm notifications instantly out to our email.
- **AWS CLI / CloudShell:** The command-line tools used to interact with AWS and test the system.

3. Step-by-Step Implementation

Step 1: Storing the Secret

First, we created a sensitive credential container named 'MySecretInfo' inside *AWS Secrets Manager*. To enforce robust data protection, the secret payload was encrypted using the standard *AWS Key Management Service (KMS)* managed key 'aws/secretsmanager'. (Fig. 1)

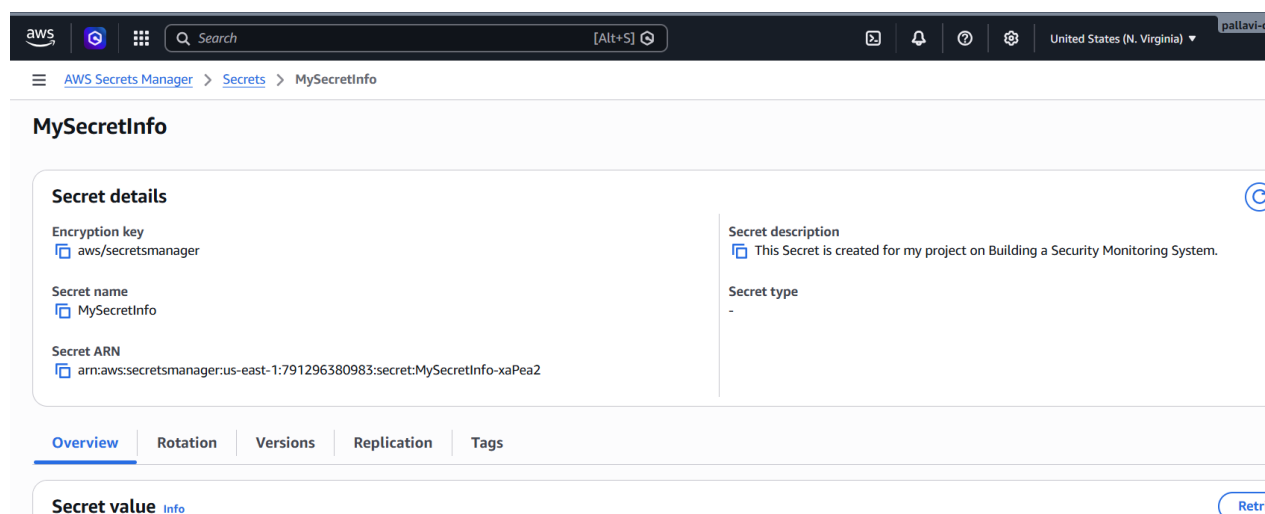


Fig. 1: Secret's details page_AWS Secrets Manager

Step 2: Setting up Audit Trail Configuration (AWS CloudTrail)

To gain visibility into credential handling, a dedicated trail was established in *AWS CloudTrail*. While standard trails capture *Management Events* (like creating or deleting a resource), this trail was explicitly configured to audit resource-level *Data Events*. This ensures that direct read-access attempts (such as API secret retrievals) are logged.

Technical Context: Read vs. Write Activities

- **Read Activities:** Operations that fetch details or list resource states without altering infrastructure (e.g., viewing an EC2 instance list or reading a secret).
- **Write Activities:** Operations that modify the environment, state, or configuration data (e.g., creating a new server, deleting a user, or updating a file).

Step 3: Sending Logs to *CloudWatch* & Creating a Filter

The raw event logs generated by CloudTrail were configured to stream dynamically into an *Amazon CloudWatch Log Group*.

Within this log group, a custom *Metric Filter* named 'GetSecretsValue' was created to evaluate unstructured log strings in real time and match patterns representing secret extraction. We configured the filter details as (Fig. 2):

- **Metric Namespace:** 'SecurityMetrics'
- **Metric Name:** 'Secret is accessed'
- **Metric Value:** '1' (This means every single time the secret is read, the counter goes up by 1).
- **Default Value:** '0' (Forces the engine to plot a baseline of zero during inactive periods, preventing missing data blocks on graphs and preserving chart contiguity).

aws CloudWatch Log management mysecretsmanager-loggroup Create metric filter

Step 3
Review and create

Log events that match the pattern you define are recorded to the metric that you specify. You can graph the metric and set alarms to notify you.

Filter name
GetSecretsValue

Filter pattern
GetSecretValue

☐ Enable metric filter on transformed logs
When enabled, metric filter will be applied to transformed logs. When disabled, metric filter will be applied to original logs.

Metric details

Metric namespace
Namespaces let you group similar metrics. [Learn more](#)
SecurityMetrics ☒ Create new
Namespaces can be up to 255 characters long; all characters are valid except for colon(:) at the start of the name.

Metric name
Metric name identifies this metric, and must be unique within the namespace. [Learn more](#)
Secret is accessed
Metric name can be up to 255 characters long; all characters are valid except for colon(:), asterisk(*), dollar(\$), and space().

Metric value
Metric value is the value published to the metric name when a Filter Pattern match occurs.
1
Valid metric values are: floating point number (1, 99.9, etc.), numeric field identifiers (\$1, \$2, etc.), or named field identifiers (e.g. \$requestSize for delimited filter pattern or \$.status for JSON-based filter pattern - dollar followed by alphanumeric and/or underscore (_) characters).

Default value - optional
The default value is published to the metric when the pattern does not match. If you leave this blank, no value is published when there is no match. [Learn more](#)
0

Unit - optional

Fig. 2: Metric Filter pattern

Step 4: Zero-Trust Alarm Implementation

A *CloudWatch Alarm* was deployed to monitor the custom metric stream. A strict *Zero-Trust rule* was applied to treat even a single access event as a potential security risk:

- **Statistic:** 'Sum' (adds up the total number of times the secret was accessed).
- **Evaluation Interval:** '1 Minute' (Checks the math every minute for the fastest response).
- **Threshold Value:** '1'.
- **Logical Condition:** Greater than or equal to ($\geq$) '1'.
 - **0 Accesses** = Baseline Operational Environment (**OK State**).
 - **1+ Accesses** = Threshold crossed / High-Priority Security Breach (**ALARM State**).

Step 5: Decoupled Email Alerting Pipeline

To get notified immediately, we set up an *AWS SNS Topic* called *SecurityAlarms*.

1. **Pub/Sub Coupling:** The *CloudWatch Alarm* acts as the publisher. The moment the alarm triggers, it broadcasts state changes directly to the *SNS Topic*.
2. **The Subscriber:** An administrative email address was bound to the *Topic* to receive instant message routing.
3. **Security Handshake:** AWS enforces a mandatory *Subscription Confirmation* workflow to guarantee that data endpoint delivery permissions are authorized. (Fig. 3)

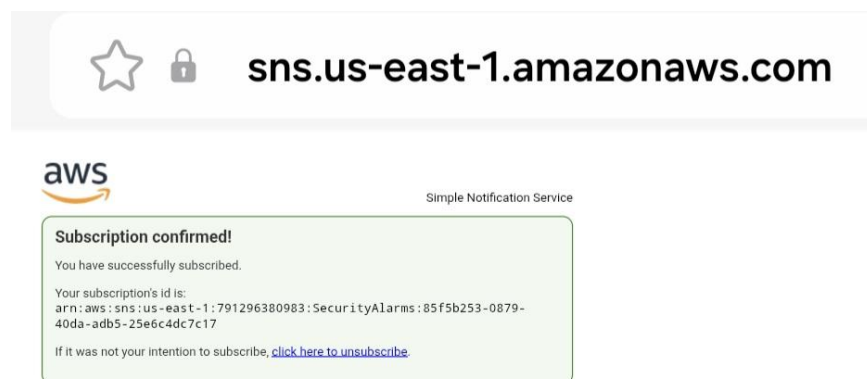


Fig. 3: Subscription confirmed AWS SNS

4. Testing the System

To check if our security pipeline works from end to end, we simulated an access event. We opened *AWS CloudShell* and ran the command-line instruction to read our secret as shown below:

`aws secretsmanager get-secret-value --secret-id "MySecretInfo" --region us-east-1`
(Fig. 4).



```

aws
[Search] [Alt+S]
United States (N. Virginia) pallavi-

CloudShell
us-east-1 +

~ $ aws secretsmanager get-secret-value --secret-id "MySecretInfo" --region us-east-1
{
  "ARN": "arn:aws:secretsmanager:us-east-1:791296380983:secret:MySecretInfo-xaPea2",
  "Name": "MySecretInfo",
  "VersionId": "ceddb6fc-e3d1-4326-9374-ebd1ba2a47a7",
  "SecretString": "{\"This Secret is\":\"secret must be kept secret\"}",
  "VersionStages": [
    "AMSCURRENT"
  ],
  "CreateDate": "2026-06-02T00:06:09.363000+00:00"
}
~ $

```

Fig. 4: Retrieving the secret using the AWS CLI

What happened behind the scenes:

- The command ran successfully and displayed the secret content on the screen.
- *AWS CloudTrail* caught this action as a resource-level *Data Event*.
- The event log streamed into *CloudWatch Logs*, triggered our custom metric filter, and changed the counter from 0 to 1.

5. Troubleshooting & Lessons Learned

During our first test, the email alert did not arrive right away. To fix this, we performed a structured check across the entire setup pipeline control plane:

CloudTrail Configurations → Log Delivery Status → Metric Filters → Alarm Configurations → SNS Notification.

This troubleshooting step helped us find and fix three important details:

- **Metric Aggregation Logic:** Initially, the alarm statistic was set to *Average*. An average calculation can dilute a quick, single access event over time, causing the alarm to miss it. Changing this setting to *Sum* ensures that every single event is fully calculated.
- **Time Optimization:** Changing the evaluation timeframe from *5 minutes* down to *1 minute* cut down the delay significantly, giving us near real-time alerts.
- **Chart Baseline:** Explicitly setting the *Default Value* = 0 keeps our dashboards clean and continuous, making it easy to see exactly when traffic drops back to normal.

Final Success and Verification

Once we saved these pipeline updates (switching to *Sum* and a *1 Minute* timeframe), we tested the system again. The security engine performed perfectly:

- Within the 1-minute window, the *CloudWatch Alarm* engine picked up the activity spike and successfully turned to the *ALARM* state. (Fig. 5)

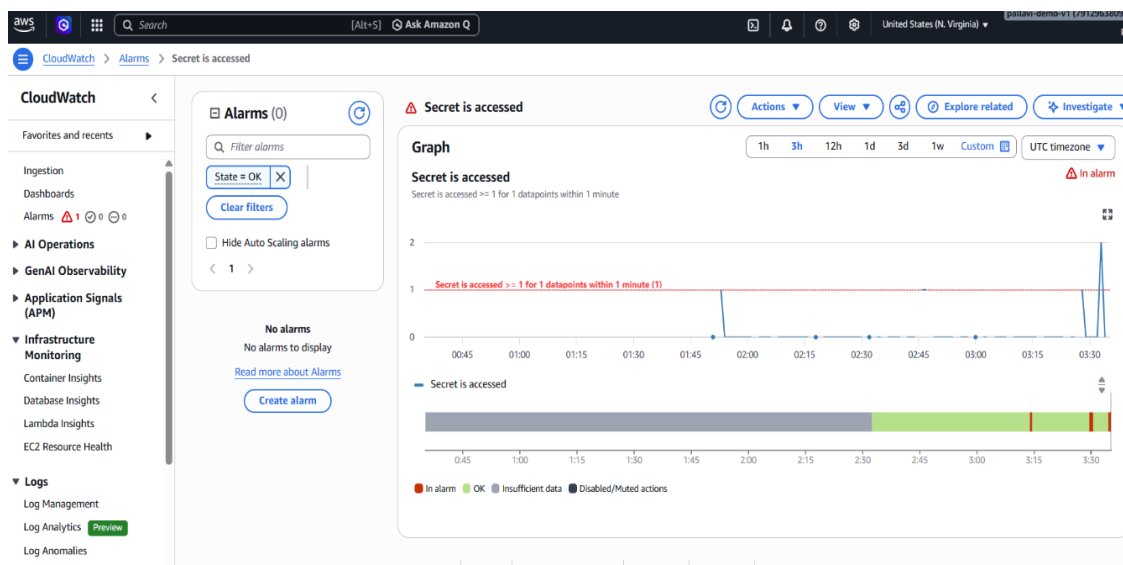


Fig. 5: CloudWatch alarm in its 'in alarm' state

- The alarm passed the message to our *AWS SNS topic*, which delivered the automated alert email containing the full technical log data straight to our inbox. (Fig. 6)

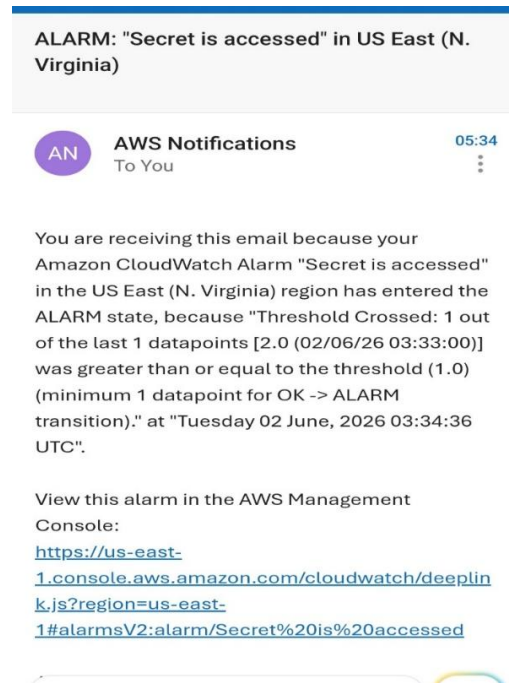


Fig. 6: Alarm notification email